

Go: The Server Language

Go, also known as Golang, is an open source programming language that makes it easy to build simple, reliable and efficient software. It is a highly performant, smoothly compiled, portable, modular, object-oriented, statically typed, and garbage collected language.



Have you ever been confused about which programming language to choose while starting a new project? I must confess that I faced the situation recently. My bare minimum requirement was to create simple yet efficient Web servers which could be developed and started within minutes. I considered many languages including Java, JS, C++ and C, but in the end I decided to go with a language that was new to me — the Go language, also known as Golang.

The next obvious question is, “Why Go?” The answer is simple: The compilation time and verbosity of Go projects is much less when compared to other languages. Since it is a statically typed language, most of the errors are immediately caught at compilation time. It also has inbuilt concurrency support, which makes its code safe to run in a multi-threaded environment. The concept of Go routine (a lightweight process), channels and select statements help users in implementing concurrent programs and utilise the multi-core system to its maximum.

From the point of view of adaptability and supportability, Go is pretty popular as it is backed and developed by Google engineers Ken Thomson (previously developed C, UNIX, UTF-8), Rob Pike (previously

developed UNIX, UTF-8), Robert Griesemer (previously developed JVM, HotSpot) and is widely used in many big companies like YouTube, Apple, Dropbox, GitHub, Intel, and so on. Developed in 2007, it is more than a decade old now. In 2009, it was launched as an open source programming language.

In this article, we will be developing a Web server that can accept a GET request and serve a response. The Web server is a program running on a computer with static/dynamic IP and a specified port, and it listens to an HTTP request sent by the client. The browser (client) sends the data to a server in the form of packets over a network, to which the server may respond with some requested data.

To develop an HTTP server all we need is a Go compiler and a text/code editor. The Golang compiler can be downloaded from the Go official binary distribution, which is available at <https://golang.org/dl/> for FreeBSD (release 10-STABLE and above), Linux, MacOS (10.11 and above), and Windows operating systems for both 32-bit (386) and 64-bit (amd64) x86 processor architectures.

Once downloaded, the complete processes for installation and testing the installation can be found at <https://golang.org/doc/install#install>. If the project involves dependencies on other Go projects, modules need to be used as required by the default dependency management system from Go 1.13 onwards.

The Go modules are collections of Go packages stored in a file tree with

the *go.mod* file at its root. The *go.mod* file contains entries that are also the import path used for the root directory and its dependency requirements.

Up next, for writing the code, we need a text editor like textpad, notepad or any IDE. I will be using VSCODE, a free and open source code editor that can be downloaded from <https://code.visualstudio.com/download>.

To support networking, Golang has a package named ‘net’, which contains the http package used to create the HTTP/ HTTPS server and make client requests. The net package (<https://golang.org/src/net/net.go>) provides an interface to support network I/O like TCP/ IP, UDP, domain name resolution, and emails. The http package has a *ListenAndServe* method, which starts the server and listens to the incoming requests.

This method accepts two arguments:

1. *address*: This is the combination of the IP address and the port of the machine in a string format. If only the port is provided, such as ‘:9000’, then this port is accessible by all the IP addresses of the system.
2. *handler*: This argument is of the type Handler interface. This uses the *serveHTTP* method, which is internally called whenever the *ListenAndServe* method is executed.
3. The *serveHTTP* method is of format *ServeHTTP (res http.ResponseWriter, req *http.Request)*, and is the main function that accepts the request and returns the response.

